

Table of Contents

SE 2144 Midterms	p.2
Project Specification: ServiceHub (Microservice & Health Registry)	p.2
1. System Overview & Architecture	p.2
2. Backend Architecture (ExpressJS + TypeScript)	p.2
3. Frontend Architecture (React + TypeScript)	p.4
4. Evaluation and Grading Rubric (100 Points)	p.4

SE 2144 Midterms

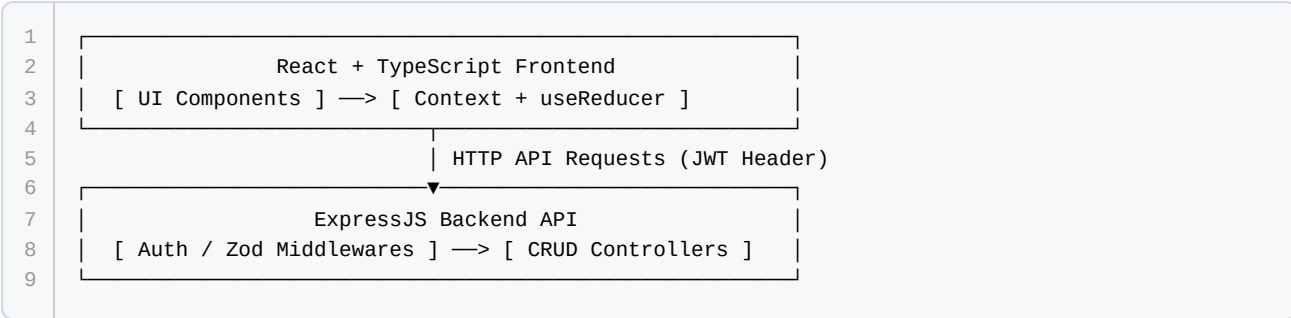
Project Specification: ServiceHub (Microservice & Health Registry)

Target Stack: ExpressJS + TypeScript (Backend) | React + TypeScript (Frontend)

Core Stack Focus: Standard Zod Validation, JWT Auth, Express Middlewares, Full CRUD, React Context + useReducer, DB.

1. System Overview & Architecture

ServiceHub is an internal developer registry for tracking microservice endpoints, health statuses, and deployment environments (DEVELOPMENT, STAGING, PRODUCTION). Developers can register new services, inspect service configurations, update operational health statuses, and decommission services.

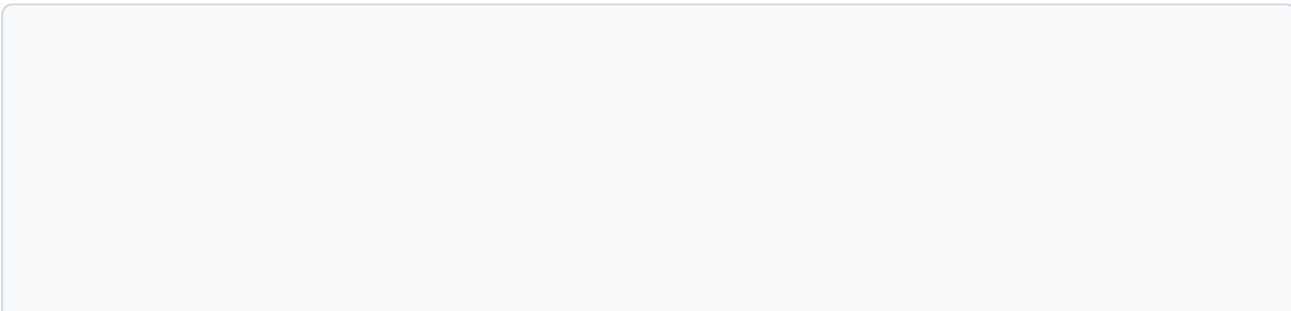


Technology Stack Matrix

Layer	Technologies & Libraries
Backend Runtime	"Node.js (v18+), ExpressJS, TypeScript"
Security & Validation	"jsonwebtoken (JWT), bcryptjs, zod"
Frontend Framework	"React (v18+ with Vite), TypeScript"
State Management	React Context API + useReducer Hook

2. Backend Architecture (ExpressJS + TypeScript)

Data Models (`src/types/index.ts`)



```

1  export interface User {
2      id: string;
3      email: string;
4      passwordHash: string;
5      role: 'DEVELOPER' | 'LEAD';
6  }
7
8  export type Environment = 'DEVELOPMENT' | 'STAGING' | 'PRODUCTION';
9  export type ServiceStatus = 'HEALTHY' | 'DEGRADED' | 'DOWN';
10
11 export interface Microservice {
12     id: string;
13     name: string;
14     endpointUrl: string;
15     environment: Environment;
16     status: ServiceStatus;
17     version: string;
18     ownerEmail: string;
19     createdAt: string;
20 }

```

Validation

When creating a Microservice:

- **name** (string): 3-60 (inclusive) characters long
- **endpointUrl** (string)
- **environment** (string): one of the following (['DEVELOPMENT', 'STAGING', 'PRODUCTION'])
- **status** (string): one of the following (['HEALTHY', 'DEGRADED', 'DOWN'])
- **version** (string)

When updating a Microservice:

- all fields are optional
- only update the fields that are included in the update schema body

When deleting a Microservice:

- only an id is required to delete

API Route Endpoint Matrix

Method	Endpoint	Protection	Middleware	Action Description
POST	/api/auth/login	Public	Standard Validation	Authenticates user and returns JWT token
POST	/api/services	Protected	"authenticateJWT"	validate(createServiceSchema)"
GET	/api/services	Protected	authenticateJWT	READ (List): Fetches all microservices
PATCH	/api/services/:id	Protected	"authenticateJWT"	validate(updateServiceSchema)"
DELETE	/api/services/:id	Protected	"authenticateJWT"	validate(deleteServiceSchema)"

3. Frontend Architecture (React + TypeScript)

State & Actions (`src/types/index.ts`)

```

1  export interface State {
2    user: { id: string; email: string; role: string } | null;
3    token: string | null;
4    services: Microservice[];
5    selectedEnvironment: Environment | 'ALL';
6    loading: boolean;
7    error: string | null;
8  }
9
10 export type Action =
11   | { type: 'SET_AUTH'; payload: { user: any; token: string } }
12   | { type: 'LOGOUT' }
13   | { type: 'SET_ENV_FILTER'; payload: Environment | 'ALL' }
14   | { type: 'FETCH_SERVICES_SUCCESS'; payload: Microservice[] }
15   | { type: 'CREATE_SERVICE_SUCCESS'; payload: Microservice }
16   | { type: 'UPDATE_SERVICE_SUCCESS'; payload: Microservice }
17   | { type: 'DELETE_SERVICE_SUCCESS'; payload: string }
18   | { type: 'SET_ERROR'; payload: string | null };

```

4. Evaluation and Grading Rubric (100 Points)

Category	Points	Specific Requirements
1. Express Backend Setup	20 pts	"Structured controller-route setup with working REST handlers (POST, GET, PATCH, DELETE)."
2. Zod Validation Middleware	20 pts	"Custom middleware parses req.body/req.params against standard Zod schemas (string(), enum()) and returns structured 400 errors."
3. JWT Authentication	15 pts	"Auth middleware validates Bearer , rejects unauthenticated requests with 401/403 status codes, and attaches req.user."
4. Full API CRUD Operations	15 pts	"Functional implementation of all 4 CRUD actions (POST, GET, PATCH, DELETE) with standard HTTP status codes."
5. React Context + useReducer	20 pts	"ServiceContext configured with pure reducer functions managing auth, CRUD actions, and environmental filtering without direct state mutation."
6. Frontend Integration & Dispatch	10 pts	"Components perform authenticated API calls with JWT headers, handle UI errors, and dispatch reducer actions upon success."